Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using content-based filtering approach	
Experiment No: 11	**Date: 01-04-2026**	**Enrolment No: 92410133038**

Aim: To recommend an item to the user using content-based filtering approach

IDE: Google Colab

Theory:

There are a lot of applications where websites collect data from their users and use that data to predict the likes and dislikes of their users. This allows them to recommend the content that they like. Recommender systems are a way of suggesting or similar items and ideas to a user's specific way of thinking.

Recommender System is different types:

- **Collaborative Filtering:** Collaborative Filtering recommends items based on similarity measures between users and/or items. The basic assumption behind the algorithm is that users with similar interests have common preferences.
- **Content-Based Recommendation:** It is supervised machine learning used to induce a classifier to discriminate between interesting and uninteresting items for the user.

Content-Based Recommendation System: Content-Based systems recommends items to the customer similar to previously high-rated items by the customer. It uses the features and properties of the item. From these properties, it can calculate the similarity between the items.
In a content-based recommendation system, first, we need to create a profile for each item, which represents the properties of those items. From the user profiles are inferred for a particular user. We use these user profiles to recommend the items to the users from the catalog.

Item profile:
In a content-based recommendation system, we need to build a profile for each item, which contains the important properties of each item. For Example, If the movie is an item, then its actors, director, release year, and genre are its important properties, and for the document, the important property is the type of content and set of important words in it.
Let's have a look at how to create an item profile. First, we need to perform the TF-IDF vectorizer, here TF (term frequency) of a word is the number of times it appears in a document and The IDF (inverse document frequency) of a word is the measure of how significant that term is in the whole corpus.

User profile:
The user profile is a vector that describes the user preference. During the creation of the user's profile, we use a utility matrix that describes the relationship between user and item. From this information, the best estimate we can decide which item the user likes, is some aggregation of the profiles of those items.
Advantages and Disadvantages:

- **Advantages:**
    - No need for data on other users when applying to similar users.
    - Able to recommend to users with unique tastes.
    - Able to recommend new & popular items

- Explanations for recommended items.
- **Disadvantages**:
  - Finding the appropriate feature is hard.
  - Doesn't recommend items outside the user profile.

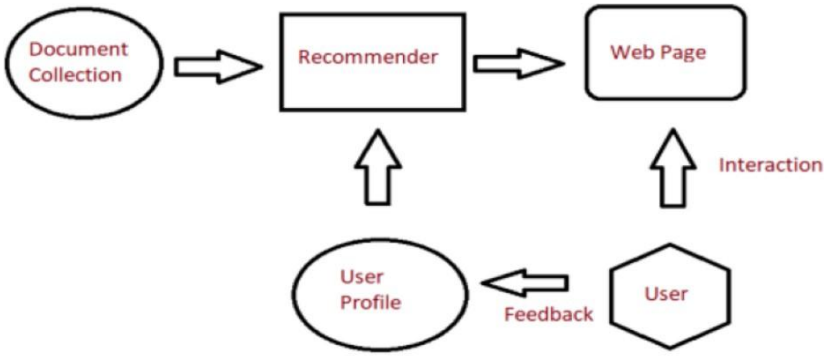

Fig: Recommender System

Pre Lab Exercise:

1. When can you use content-based recommendation system approach?

   Use a content-based recommendation system when you want to suggest items based on a user's past preferences and item features. It is useful when user history is available, item attributes (like genre or category) are known, and personalization is important. It works well even with fewer users and helps provide recommendations without depending on others' data

2. Write advantages of content-based filtering approach

   - Personalized recommendations
     Suggests items based on individual user preferences
   - No need for other users' data
     Works independently without relying on other users.

- Handles new users better (cold start)
  Can recommend items using initial user preferences
- No popularity bias
  Recommends items based on interest, not trends.
- Explainable results
  Easy to justify recommendations (based on similar features).
- Works well with structured item data
  Effective when item features like genre, category, or keywords are available.

3. Write disadvantages of content-based filtering approach

- Limited recommendations
  Suggests only similar items, lacks variety
- Requires detailed item features
  Needs well-defined attributes for accurate results
- Over-specialization
  User may not discover new or different types of items.
- Cold start for items
  New items cannot be recommended until features are analyzed.
- Feature extraction is complex
  Difficult to extract features for some data (e.g., images, videos).

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using content-based filtering approach	
Experiment No: 11	**Date: 01-04-2026**	**Enrolment No: 92410133038**

Program (Code):

To be attached with

```
# @title

import pandas as pd

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics.pairwise import cosine_similarity

df=pd.read_csv("/content/sample_data/movies.csv")

#preprocessing step

df["genres"]=df["genres"].fillna("")

df["overview"]=df["overview"].fillna("")

df["content"]=df["genres"]+" "+df['overview']

#TF-IDF Vectorizer

tfidf=TfidfVectorizer(stop_words="english")

tfidf_matrix=tfidf.fit_transform(df["content"])

cosine_sim=cosine_similarity(tfidf_matrix,tfidf_matrix)

#droping the duplicates

indices=pd.Series(df.index,index=df["original_title"]).drop_duplicates()

def recommendation_movie(title,topn=10):

  if title not in indices:

    return "Movie is not found!"

  idx=indices[title]

  sim_scores=list(enumerate(cosine_sim[idx]))

  sim_scores=sorted(sim_scores,key=lambda x:x[1],reverse=True)

  sim_scores=sim_scores[1:topn+1]
```

```
    movie_indices=[i[0] for i in sim_scores]

    return df["original_title"].iloc[movie_indices]

recommendation_movie("Man of Steel")
```

Results:

To be attached with

	original_title
223	The Chronicles of Riddick
220	Prometheus
2375	Midnight Special
539	Titan A.E.
1275	Sunshine
4401	The Helix... Loaded
1934	Say It Isn't So
200	The Hunger Games: Mockingjay - Part 1
1044	Journey to the Center of the Earth
90	The Polar Express

dtype: object

Observation:

The code provided implements a content-based recommendation system utilizing Natural Language Processing methodologies. The initial step is to load a set of movies and carry out preprocessing by filling gaps in genres and overview columns with some values. Then these two characteristics are concatenated into a single textual feature called content, which denotes the descriptive information of every film.

Then, the program performs TF-IDF (Term FrequencyInverse Document Frequency) using scikit-learn to change the textual content to a numerical feature matrix. This change is useful in pointing out the importance of words in the description of every movie while lessening the effect of words that occur very frequently.

Immediately after vectorization, cosine similarity is calculated between all pairs of movie vectors to determine the similarity between movies. Similarity scores lie between 0 and 1, where larger scores denote a higher similarity in content.

A kind of index where movie titles act as keys has been made to enabling us to get their corresponding places in the dataset very fast. The recommendation_movie() function first takes a movie title as an argument, and then it finds similarity scores of the given movie with all other movies, sorts them in descending order, and outputs top N most similar movies (excluding the input movie itself).

Post Lab Exercise:

1. What changes you feel can be done, to improve the system that you have designed, as a part of in-house lab exercise.

   There are many ways in which the system can be enhanced, including the addition of more features, the use of advanced NLP methods, transitioning from TF-IDF to word embeddings, merging with collaborative filtering for improved personalization, and tuning for high performance on large datasets, among others.

2. Recommend top-10 “series” using IMDB movie dataset (Link: https://www.kaggle.com/datasets/harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows). Attach the code with output. Also, write your observation for the same.